

Dá para fazer **muita coisa útil com Regex em Python antes de mandar a transcrição para uma LLM**. No seu atendimento real, por exemplo, conseguimos extrair automaticamente protocolo, CEP, data de nascimento, idade, telefone, produto, intenção do cliente, identificação dos participantes e até verificar se determinadas etapas do script aconteceram.

Na sua transcrição, o cliente solicita **“boleto de quitação do meu cartão consignado”**, fornece CEP, nome da mãe e data de nascimento; no final, o atendente informa que o setor responsável já encerrou o atendimento e fornece outro telefone.

1. Regex para extrair o protocolo

Na conversa aparece:

Esta ligação gerou o protocolo 2606648481.

Podemos procurar exatamente uma sequência de 10 números:

```
import re

padrao = r"\b\d{10}\b"

resultado = re.findall(padrao, texto)

print(resultado)
```

Resultado:

```
['2606648481']
```

Melhor ainda:

```
protocolo = re.search(
    r"protocolo\s+(\d{10})",
    texto,
    re.IGNORECASE
)

if protocolo:
    print(protocolo.group(1))
```

Resultado:

```
2606648481
```

Isso poderia alimentar automaticamente uma coluna:

```
PROTOCOLO_ATENDIMENTO = 2606648481
```

2. Detectar o CEP

Na transcrição:

```
#Atendente: Confirma para mim o CEP.
#Cliente: 04864220.
```

Regex:

```
cep = re.search(r"\b\d{8}\b", texto)
if cep:
    print(cep.group())
```

Resultado:

```
04864220
```

No banco:

```
CEP = 04864220
```

Mas veja uma coisa importante: Regex detecta que existe um número de oito dígitos. Para aumentar a precisão, podemos considerar também o contexto:

```
cep = re.search(
    r"CEP.*?#Cliente:\s*(\d{8})",
    texto,
    re.IGNORECASE | re.DOTALL
)
print(cep.group(1))
```

Aí estamos dizendo:

encontre a palavra CEP e procure a resposta seguinte do cliente.

Isso reduz muito falso positivo.

3. Detectar data de nascimento

Na conversa:

```
#Atendente: Data de nascimento da senhora?
#Cliente: 23/10/1966.
```

Regex:

```
data = re.search(
    r"\b\d{2}/\d{2}/\d{4}\b",
    texto
)
```

```
print(data.group())
```

Resultado:

```
23/10/1966
```

Podemos armazenar:

```
DATA_NASCIMENTO = 23/10/1966
```

Ou capturar especificamente depois da pergunta:

```
data = re.search(
    r"Data de nascimento.*?#Cliente:\s*(\d{2}/\d{2}/\d{4})",
    texto,
    re.IGNORECASE | re.DOTALL
)

print(data.group(1))
```

4. Detectar idade

O diálogo contém:

```
#Atendente: Atualmente está com quantos anos?
```

```
#Cliente: 59.
```

Podemos fazer:

```
idade = re.search(
    r"quantos anos.*?#Cliente:\s*(\d{1,3})",
    texto,
    re.IGNORECASE | re.DOTALL
)

if idade:
    print(idade.group(1))
```

Resultado:

```
59
```

E criar:

```
IDADE_INFORMADA = 59
```

5. Detectar telefone informado pelo atendente

No final:

```
#Atendente: É o 3004-5300.
```

Regex:

```
telefone = re.search(
    r"\b\d{4}-\d{4}\b",
    texto
)

print(telefone.group())
```

Resultado:

```
3004-5300
```

No banco:

```
TELEFONE_INFORMADO = 3004-5300
```

Podemos suportar outros formatos também:

```
padrao_telefone = r"""
(?:
    \ (? \d{2} \) ? \s*
)?
\d{4,5}
[-\s]?
\d{4}
"""

telefones = re.findall(
    padrao_telefone,
    texto,
    re.VERBOSE
)
```

Assim pegaria coisas como:

```
3004-5300
11 3004-5300
(11) 3004-5300
1199999-9999
(11) 99999-9999
```

6. Identificar quem é cliente e quem é atendente

Aqui Regex fica extremamente útil.

Sua transcrição já tem:

```
#Atendente:
#Cliente:
```

Podemos separar todas as falas:

```
falas_atendente = re.findall(
    r"#Atendente:\s*(.*?) (?:\n\s*#Cliente:|\Z)",
    texto,
    re.DOTALL
)

falas_cliente = re.findall(
    r"#Cliente:\s*(.*?) (?:\n\s*#Atendente:|\Z)",
    texto,
    re.DOTALL
)
```

Depois:

```
print("Falas atendente:", len(falas_atendente))
print("Falas cliente:", len(falas_cliente))
```

Isso permite montar coisas como:

```
TOTAL_TURNOS
TURNOS_CLIENTE
TURNOS_ATENDENTE
TEXTO_CLIENTE
TEXTO_ATENDENTE
```

7. Extrair somente tudo que o cliente falou

Imagine mandar para uma análise de sentimento somente as falas do cliente.

```
falas_cliente = re.findall(
    r"#Cliente:\s*(.*?) (?:\n\s*#Atendente:|\Z)",
    texto,
    flags=re.DOTALL
)

texto_cliente = " ".join(
    fala.strip()
)
```

```

        for fala in falas_cliente
    )

print(texto_cliente)

```

Você obteria algo semelhante a:

Oi, meu nome é Estéia.

Eu precisava do boleto de quitação do meu cartão consignado.

Isso, do meu cartão consignado.

04864220.

Lourdes Fonseca Pinto.

23/10/1966.

59.

Pode ser, sim. Pode falar para mim.

Tá bom. Obrigada, viu?

Isso é muito interessante porque sua LLM não precisa receber necessariamente a transcrição inteira para todas as tarefas.

8. Extrair somente o que o atendente falou

Mesma ideia:

```

falas_atendente = re.findall(
    r"#Atendente:\s*(.*?) (?=\n\s*#Cliente:|\Z)",
    texto,
    flags=re.DOTALL
)

texto_atendente = " ".join(
    fala.strip()
    for fala in falas_atendente
)

```

Agora você pode analisar separadamente:

```

POSTURA_ATENDENTE
SCRIPT_ATENDENTE
CORDIALIDADE
TRANSFERÊNCIA
ORIENTAÇÃO
DESPEDIDA

```

9. Detectar o produto bancário

Nesse atendimento aparece repetidamente:

cartão consignado

Podemos criar um dicionário:

```
produtos = {
    "cartao_consignado": r"\bcart[aão]o consignado\b",
    "emprestimo_consignado": r"\bempr[eé]stimo consignado\b",
    "cartao_credito": r"\bcart[aão]o de cr[eé]dito\b",
    "financiamento": r"\bfinanciamento\b",
    "conta_corrente": r"\bconta corrente\b"
}
```

E executar:

```
produto_detectado = None

for produto, padrao in produtos.items():
    if re.search(padrao, texto, re.IGNORECASE):
        produto_detectado = produto
        break

print(produto_detectado)
```

Resultado:

cartao_consignado

Banco:

PRODUTO = CARTAO_CONSIGNADO

10. Detectar o motivo do contato

O cliente disse:

Eu precisava do boleto de quitação do meu cartão consignado.

Podemos criar regras:

```
motivos = {
    "quitacao": r"\bquita[cç][aão]\b",
    "segunda_via": r"\bsegunda via\b",
    "boleto": r"\bboleto\b",
    "cancelamento": r"\bcancel(?:ar|amento)\b",
    "portabilidade": r"\bportabilidade\b",
    "reclamacao": r"\breclama[cç][aão]\b"
}
```

Executando:

```
motivos_encontrados = []

for motivo, padrao in motivos.items():
    if re.search(padrao, texto, re.IGNORECASE):
        motivos_encontrados.append(motivo)

print(motivos_encontrados)
```

Nesse atendimento:

```
['quitacao', 'boleto']
```

Podemos transformar em:

```
MOTIVO_PRINCIPAL = QUITACAO
OBJETO = BOLETO
PRODUTO = CARTAO_CONSIGNADO
```

Agora começamos a chegar em algo muito poderoso:

```
INTENCAO_CLIENTE =
SOLICITAR_BOLETO_QUITACAO_CARTAO_CONSIGNADO
```

11. Verificar se houve validação de segurança

Essa é uma aplicação excelente de Regex para monitoria.

Seu atendimento contém:

```
Confirma para mim o CEP.
Nome da mãe?
Data de nascimento da senhora?
```

Podemos criar:

```
validacoes = {
    "cep": r"\bCEP\b",
    "nome_mae": r"\bnome da m[aã]e\b",
    "data_nascimento": r"\bdata de nascimento\b",
    "cpf": r"\bCPF\b",
    "rg": r"\bRG\b"
}
```

Rodando:

```
resultado = {}

for campo, padrao in validacoes.items():
    resultado[campo] = bool(
```



```

        re.search(
            padrao,
            texto,
            re.IGNORECASE
        )
    )

print(resultado)

```

Resultado esperado:

```

{
    "cep": True,
    "nome_mae": True,
    "data_nascimento": True,
    "cpf": False,
    "rg": False
}

```

Isso já gera uma espécie de checklist automático.

12. Monitoria automática do script

Podemos fazer algo ainda melhor.

```

checklist = {
    "saudacao": r"boa (?:tarde|dia|noite)",
    "identificacao_banco": r"Banco Daycoval",
    "protocolo": r"protocolo\s+\d+",
    "cep": r"\bCEP\b",
    "nome_mae": r"nome da m[aã]e",
    "data_nascimento": r"data de nascimento",
    "desculpas": r"desculp",
    "despedida": r"excelente (?:dia|tarde|noite)"
}

resultado = {}

for item, regex in checklist.items():

    resultado[item] = (
        "SIM"
        if re.search(regex, texto, re.IGNORECASE)
        else "NAO"
    )

print(resultado)

```

Nesse atendimento teríamos algo próximo de:

SAUDACAO	SIM
IDENTIFICACAO_BANCO	SIM
PROTOCOLO	SIM
CEP	SIM
NOME_MAE	SIM

DATA_NASCIMENTO	SIM
DESCULPAS	SIM
DESPEDIDA	SIM

E isso sem gastar **um token de LLM**.

13. Detectar se houve transferência

Observe algo interessante no atendimento:

Eu não vou encaminhar a senhora para lá nem transferir

Uma Regex ingênua seria:

```
if re.search(r"transfer", texto, re.IGNORECASE):
    print("Houve transferência")
```

E estaria **ERRADA**.

Porque encontrou:

"nem transferir"

Esse é justamente o tipo de problema que precisamos aprender a resolver.

Podemos procurar negação:

```
transferencia_negada = re.search(
    r"(?:n[aã]o|nem){0,30}transfer",
    texto,
    re.IGNORECASE
)
```

Resultado:

TRANSFERENCIA_REALIZADA = NÃO

Aqui você começa a enxergar uma diferença importante:

Regex simples

transfer

diz:

Achei a palavra transferência.

Regex contextual

diz:

A palavra apareceu dentro de uma negação.

14. Detectar problema de horário

Na transcrição:

funciona só até às 17 horas

e:

já que passou do horário

Podemos detectar:

```
horario = re.search(
    r"at[eé]\s+[àa]s?\s*(\d{1,2})\s*horas",
    texto,
    re.IGNORECASE
)

if horario:
    print(horario.group(1))
```

Resultado:

17

E:

```
fora_horario = bool(
    re.search(
        r"passou do hor[aá]rio",
        texto,
        re.IGNORECASE
    )
)
```

Então:

```
HORARIO_LIMITE = 17:00
FORA_HORARIO = SIM
```

15. Detectar resolução

Nesse atendimento temos uma situação interessante.

O atendente disse:

Peço desculpas por não ter conseguido ajudar.

Uma regra determinística poderia procurar:

```
padroes_nao_resolvido = [
    r"n[ã]o (?consegui|consequimos) ajudar",
    r"n[ã]o consigo resolver",
    r"n[ã]o foi poss[ií]vel",
    r"entre em contato amanhã"
]
```

Código:

```
nao_resolvido = any(
    re.search(padrao, texto, re.IGNORECASE)
    for padrao in padroes_nao_resolvido
)

print(nao_resolvido)
```

Resultado:

True

Poderíamos criar:

```
RESOLVIDO_REGEX = NÃO
```

Mas aqui eu faria uma distinção importante no seu projeto:

REGEX:
identifica evidências.

LLM:
interpreta a resolutividade.

Por exemplo:

EVIDENCIA:
"não ter conseguido ajudar"

EVIDENCIA:
"entrar em contato amanhã"

LLM:
Atendimento não resolvido.

Essa arquitetura é muito mais segura.

16. Detectar pedido de desculpas

Aqui temos duas ocorrências:

Senhora, desculpa a demora

e:

Peço desculpas por não ter conseguido ajudar.

Regex:

```
desculpas = re.findall(
    r"\b(?:desculpa|desculpas|desculpe)\b",
    texto,
    re.IGNORECASE
)

print(desculpas)
```

Teríamos duas ocorrências.

Podemos criar:

```
PEDIDO_DESCULPAS = SIM
QTD_PEDIDOS_DESCULPAS = 2
```

17. Detectar cordialidade objetiva

Regex não deve decidir sozinho:

```
ATENDENTE FOI CORDIAL = 10
```

Isso seria frágil.

Mas pode encontrar sinais:

```
expressoes_cordiais = [
    r"boa tarde",
    r"por nada",
    r"por favor",
    r"obrigad[oa]",
    r"excelente tarde",
    r"desculp"
]
```

Código:

```
cordialidade_evidencias = []

for padrao in expressoes_cordiais:

    encontrados = re.findall(
        padrao,
        texto,
        re.IGNORECASE
    )
```

```
cordialidade_evidencias.extend(encontrados)
```

Depois você envia para a LLM:

Evidências encontradas:

- Boa tarde
- Desculpa a demora
- Por nada
- Peço desculpas
- Excelente tarde

A LLM analisa a qualidade da cordialidade.

Muito melhor que entregar os 2.000 caracteres inteiros sem preparação.

18. Detectar confirmação do cliente

Temos:

Isso, do meu cartão consignado.

e:

Pode ser, sim.

Regex:

```
confirmacoes = re.findall(
    r"\b(?:sim|isso|correto|confirmando|pode ser)\b",
    texto,
    re.IGNORECASE
)
```

Você poderia obter:

```
isso
pode ser
sim
```

Aplicações:

```
CONFIRMACAO_CLIENTE
ACEITE
CONFIRMACAO_DE_PRODUTO
CONFIRMACAO_DE_PROCEDIMENTO
```

19. Identificar nomes

Temos:

meu nome é Estéia

e:

Lourdes Fonseca Pinto

Mas cuidado: detectar nomes exclusivamente com Regex é mais difícil.

O nome do cliente pode ser extraído devido ao contexto:

```
nome_cliente = re.search(
    r"meu nome [eé]\s+([A-Za-zÀ-ÿ]+)",
    texto,
    re.IGNORECASE
)

print(nome_cliente.group(1))
```

Resultado:

Estéia

Já o nome da mãe:

```
nome_mae = re.search(
    r"Nome da m[aã]e\?.*?#Cliente:\s*([A-Za-zÀ-ÿ ]+)",
    texto,
    re.IGNORECASE | re.DOTALL
)

print(nome_mae.group(1).strip())
```

Resultado:

Lourdes Fonseca Pinto

20. Mas há uma aplicação ainda mais importante: mascarar dados pessoais

Antes de mandar a conversa para uma LLM externa, poderíamos fazer:

Entrada:

```
protocolo 2606648481
CEP 04864220
Lourdes Fonseca Pinto
23/10/1966
3004-5300
```

Depois:

```
protocolo [PROTOCOLO]
CEP [CEP]
[NOME]
[DATA_NASCIMENTO]
[TELEFONE]
```

Exemplo:

```
texto_anonimizado = texto

texto_anonimizado = re.sub(
    r"\b\d{10}\b",
    "[PROTOCOLO]",
    texto_anonimizado
)

texto_anonimizado = re.sub(
    r"\b\d{2}/\d{2}/\d{4}\b",
    "[DATA]",
    texto_anonimizado
)

texto_anonimizado = re.sub(
    r"\b\d{4}-\d{4}\b",
    "[TELEFONE]",
    texto_anonimizado
)
```

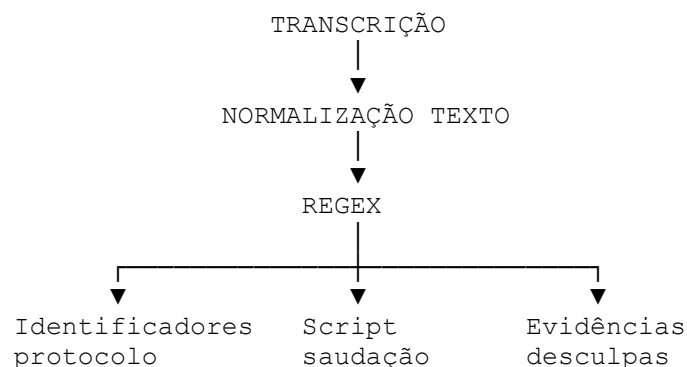
Isso pode ser uma camada importante de **pré-processamento e governança**.

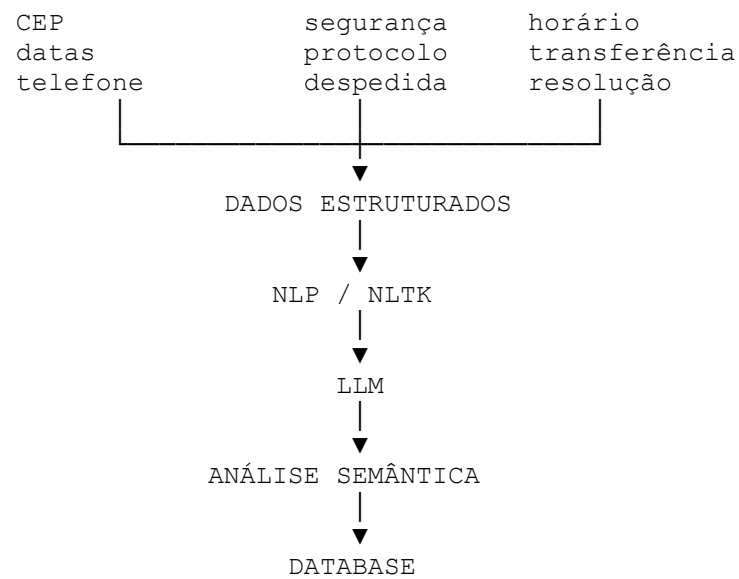
O que eu faria no seu sistema

Eu não usaria:

```
TRANSCRIÇÃO
↓
LLM
```

Eu faria:





E o resultado desse **atendimento real** poderia começar assim:

```

{
  "protocolo": "2606648481",
  "cliente": "Estéia",
  "produto": "CARTAO_CONSIGNADO",
  "motivo": "BOLETO_QUITACAO",
  "cep_identificado": True,
  "nome_mae_identificado": True,
  "data_nascimento_identificada": True,
  "idade_informada": 59,
  "telefone_orientacao": "3004-5300",
  "validacoes": {
    "cep": True,
    "nome_mae": True,
    "data_nascimento": True
  },
  "fora_horario": True,
  "horario_limite": "17:00",
  "transferencia_realizada": False,
  "orientacao_retorno": True,
  "pedido_desculpas": True,
  "indicador_nao_resolvido": True
}
  
```

Percebe o tamanho disso?

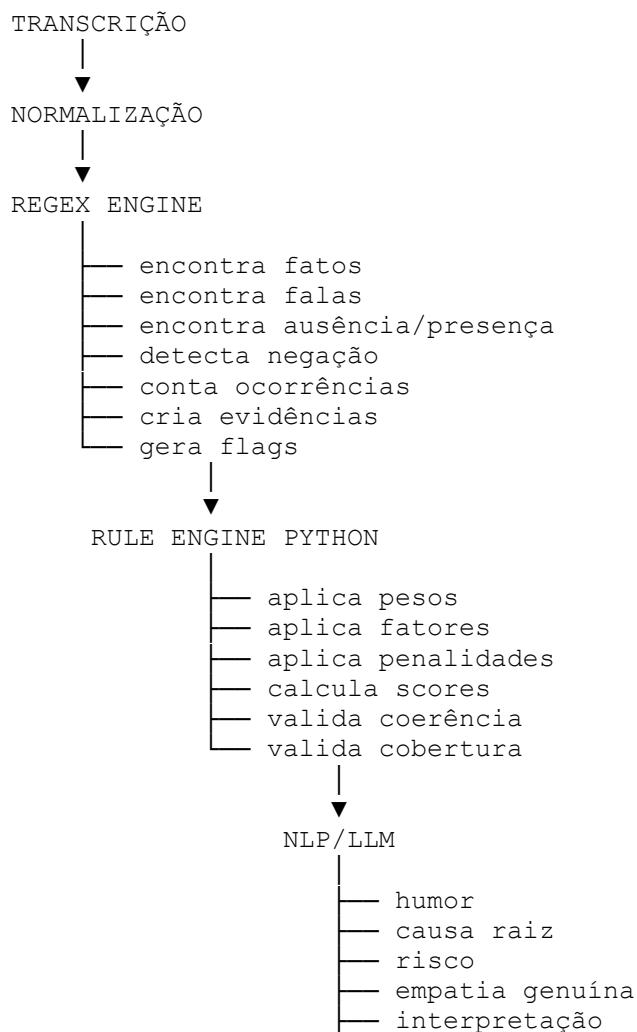
Antes de gastar **qualquer token da LLM**, Python + Regex já pode transformar uma transcrição praticamente crua em dezenas de variáveis estruturadas.

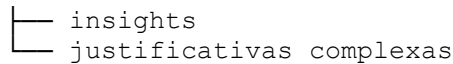
E eu iria ainda além: para seu projeto de monitoria, eu montaria uma **Camada Regex Bancária com algo entre 60 e 100 detectores**, divididos em Identificação, Segurança, Produto, Motivo, Script, Compliance, Resolução, Transferência, Reclamação, Risco, Dados pessoais, Protocolo e Evidências. A LLM receberia apenas o que realmente requer interpretação. Isso reduziria custo, tokens e alucinação consideravelmente.

Dá para usar Regex para **cobrir 100% da estrutura do documento**, mas não para **decidir semanticamente 100% das avaliações**. Essa diferença é fundamental.

O seu prompt exige, por atendimento, **14 critérios de Relacionamento, 2 de Resolutividade, 9 de CX, 7 de Inaderência, 1 Ponto Extra, 3 CES, 1 Fricção, 5 Impactos, 4 campos de Causa Raiz, Oportunidades do Operador e Falhas Operacionais**. Ele inclusive manda validar essas quantidades antes de finalizar.

Então eu montaria uma arquitetura assim:





O Regex seria o seu “coletor de provas”

O próprio documento determina que nenhuma conclusão pode existir sem evidência da transcrição, que não se pode inventar fatos e que uma informação ausente deve ser considerada como não ocorrida quando a regra determinar isso.

É exatamente aí que Regex brilha.

Por exemplo, naquele atendimento real:

"Esta ligação gerou o protocolo 2606648481"

Regex:

```
r"\bprotocolo\s*(?:n[°°o.]*)?\s*:\s*(\d{6,15})\b"
```

Resultado:

```
{
  "encontrado": True,
  "valor": "2606648481",
  "evidencia": "Esta ligação gerou o protocolo 2606648481"
}
```

Essa mesma filosofia seria aplicada a todos os critérios.

1. Identificação e indicadores gerais

Seu documento exige campos como `data_interacao`, `cpf`, `nome_cliente`, `atendente`, `produto_principal`, `motivo_contato`, `score_experiencia`, `score_operador`, `resolutividade`, `esforço`, `recontato` e `humor`.

Regex pode extrair diretamente:

```
PADROES = {
  "protocolo": r"\bprotocolo\s*(?:n[°°o.]*)?\s*:\s*(\d{6,15})\b",
  "cpf": r"\b\d{3}\.\?\d{3}\.\?\d{3}-?\d{2}\b",
  "data": r"\b\d{2}/\d{2}/\d{4}\b",
  "cep": r"\b\d{5}-?\d{3}\b",
  "telefone": r"\b(?:\d{2})?\s*\d{4,5}-?\d{4}\b",
  "idade": r"\b(\d{1,3})\s*anos\b"
}
```

Já coisas como:

```
produto_principal
motivo_contato
```

podem usar catálogo Regex:

```
PRODUTOS = {
    "Cartão consignado":
        r"\bcart[aã]o\s+consignado\b",

    "Empréstimo consignado":
        r"\bempr[eé]stimo\s+consignado\b",

    "Cartão de crédito":
        r"\bcart[aã]o\s+de\s+cr[eé]dito\b",

    "Financiamento":
        r"\bfinanciamento\b",

    "Conta corrente":
        r"\bconta\s+corrente\b"
}
```

No atendimento real:

```
"boleto de quitação do meu cartão consignado"
```

teríamos:

```
produto_principal = "Cartão consignado"
motivo_contato = "Quitação / boleto"
```

2. Relacionamento e Conduta — 14 critérios

O prompt especifica 14 critérios, incluindo tratamento pelo nome, desculpas, acolhimento, empatia, paciência, ausência de interrupção, calma, comunicação adequada, vícios de linguagem, segurança, gerundismo, clareza, objetividade e foco. A maioria pesa 3,5 e `at_rel_ling5` pesa 4,5.

Aqui eu faria detectores individuais.

`at_rel_cord1` — tratamento pelo nome

```
PADROES_NOME = [
    r"\bsenh(?:a)?\s+[A-ZÁÉÍÓÚÂÊÔÃÕÇ][a-záéíóúâêôãõç]+",
    r"\b(?:sr|sra)\.?\s+[A-ZÁÉÍÓÚÂÊÔÃÕÇ][a-záéíóúâêôãõç]+"
]
```

Também podemos primeiro capturar o nome:

```
m = re.search(
    r"(?:meu nome [ée]|quem fala [ée])\s+([A-Za-zÀ-ÿ]+)",
    texto,
    re.I
)
```

Depois procurar se o atendente utilizou esse nome.

```
usou_nome = bool(
    re.search(
        rf"#Atendente:.*\b{re.escape(nome_cliente)}\b",
        texto,
        re.I
    )
)
```

at_rel_cord2 — desculpas

```
r"\b(?:desculpa|desculpe|desculpas|peço desculpas|lamentamos?)\b"
```

No atendimento real:

```
"desculpa a demora"
```

e:

```
"Peço desculpas por não ter conseguido ajudar"
```

Regex consegue devolver:

```
{
    "classificacao": "Sim",
    "ocorrencias": 2,
    "evidencias": [
        "desculpa a demora",
        "Peço desculpas por não ter conseguido ajudar"
    ]
}
```

at_rel_cord3 — acolhimento

Podemos encontrar sinais como:

```
r"\b(?:entendo|compreendo|vou verificar|vou te ajudar|posso
ajudar|claro)\b"
```

Mas aqui há perigo.

Encontrar:

```
"entendo"
```

é objetivo.

Decidir:

"Houve acolhimento adequado"

já é parcialmente semântico.

Então o Regex entregaria:

```
{
  "sinais_acolhimento": [...],
  "quantidade": 2
}
```

e não necessariamente a decisão final.

at_rel_cord4 — empatia genuína

Esse é um ótimo exemplo de algo que **não deve ser 100% Regex**.

Podemos detectar:

```
PADROES_EMPATIA = [
  r"\b(?:entendo|compreendo)\b",
  r"\bsinto muito\b",
  r"\bimagino\b",
  r"\blamento\b",
  r"\bpeço desculpas\b"
]
```

Mas:

"Entendo."

não significa necessariamente empatia genuína.

Regex = evidência.

LLM/NLP = interpretação.

at_rel_cord6 — interrupções

Se sua transcrição tiver timestamps ou sobreposição:

```
[00:02:14] Cliente...
[00:02:15] Atendente...
```

é possível identificar sobreposição.

Sem timestamps, Regex não consegue saber se uma pessoa fisicamente interrompeu a outra.

Logo:

```
at_rel_cord6 = "Não identificado"
```

quando a evidência não existir.

Isso está perfeitamente alinhado ao seu próprio documento, que manda usar “Não identificado” ou “Não aplicável” quando não existe evidência suficiente.

3. Linguagem ofensiva

Esse pode ser quase totalmente determinístico.

```
PALAVRAS_OFENSIVAS = [
    r"\bidiota\b",
    r"\bburro\b",
    r"\bmerda\b",
    r"\bporra\b",
    r"\bincompetente\b"
]
def detectar_ofensas(texto):
    evidencias = []

    for padrao in PALAVRAS_OFENSIVAS:
        evidencias += re.findall(
            padrao,
            texto,
            re.I
        )

    return evidencias
```

Se ocorrer:

```
{
    "linguagem_ofensiva": True,
    "evidencias": [...]
}
```

E o motor de consistência pode aplicar:

```
if linguagem_ofensiva:
    humor_atendente = "Negativo"
```

Seu próprio prompt estabelece que, havendo linguagem ofensiva, `humor_atendente` não pode ser positivo.

Isso não precisa ser LLM.

4. Gerundismo

Regex é excelente aqui.

```
GERUNDISMO =
r"\b(?:vou|vamos|estarei|iremos)\s+\w+(?:ando|endo|indo)\b"
```

Pode detectar:

```
vou estar verificando
vou estar enviando
estaremos entrando em contato
vou estar solicitando
```

Exemplo:

```
ocorrencias = re.findall(
    GERUNDISMO,
    texto,
    re.I
)
```

Você já consegue alimentar diretamente:

```
at_rel_ling4
```

5. Vícios de linguagem

Também podemos contar:

```
VICIOS = [
    r"\bné\b",
    r"\btá\b",
    r"\btipo assim\b",
    r"\bbeleza\b",
    r"\bcerto?\b",
    r"\bentendeu?\b",
    r"\bokay\b"
]
```

Saída:

```
{
    "ne": 7,
    "ta": 13,
    "tipo_assim": 2
}
```


Depois você pode definir regra:

```
if total_vicios >= LIMITE:
    classificacao = "Não"
else:
    classificacao = "Sim"
```

O limite deve ser definido pela sua política de monitoria, não inventado pelo Regex.

6. Resolutividade

Seu documento possui:

```
at_resol_solic1 = Proatividade
at_resol_solic2 = Esforço do cliente
```

com peso 5 cada.

Regex pode detectar fortemente sinais de não resolução:

```
PADROES_NAO_RESOLUCAO = [
    r"\bn[ã]o consegui(?:mos)? resolver\b",
    r"\bn[ã]o consegui(?:mos)? ajudar\b",
    r"\bentre em contato novamente\b",
    r"\bentre em contato amanhã\b",
    r"\blique novamente\b",
    r"\bn[ã]o foi poss[ií]vel\b",
    r"\bn[ã]o consigo fazer\b"
]
```

No atendimento real:

"Peço desculpas por não ter conseguido ajudar."

temos uma evidência fortíssima.

Então:

```
indicador_ao_resolucao = True
```

Também aparece:

"entrar em contato amanhã"

Então:

```
recontato_explicito = True
```

A combinação pode permitir uma regra determinística:

```
if indicador_ao_resolucao and recontato_explicito:
```

```
atendimento_resolutivo = "Não"
```

7. Próximos passos

Regex pode procurar:

```
PROXIMOS_PASSOS = [
    r"\bentre em contato\b",
    r"\blique\b",
    r"\bretorne\b",
    r"\bamanhã\b",
    r"\bdeverá\b",
    r"\bpróximo passo\b",
    r"\bserá necessário\b"
]
```

No atendimento real:

"entrar em contato amanhã"

Logo:

```
proximos_passos_informados = True
```

Isso alimenta diretamente critérios do CX e falhas operacionais.

Seu documento determina inclusive que, se não foram informados próximos passos, `at_cx_classif2` não pode ser "Sim".

8. Confirmação de dados

Esse é outro excelente campo determinístico.

```
VALIDACOES = {
    "cpf":
        r"\bCPF\b",

    "cep":
        r"\bCEP\b",

    "data_nascimento":
        r"\bdata de nascimento\b",

    "nome_mae":
        r"\bnome da m[aã]e\b",

    "rg":
        r"\bRG\b"
}
```

No atendimento real:

```
{
    "cep": True,
    "nome_mae": True,
    "data_nascimento": True,
    "cpf": False,
    "rg": False
}
```

Isso alimenta:

at_cx_intro2

O documento define explicitamente que at_cx_intro2 representa confirmação de dados.

9. Repetição da solicitação

Também podemos fazer sem LLM.

Extraímos só as falas do cliente:

```
falas_cliente = re.findall(
    r"#Cliente:\s*(.*?) (?:\n#Atendente:|\Z)",
    texto,
    re.S
)
```

Normalizamos:

```
def normalizar(frase):
    frase = frase.lower()
    frase = re.sub(r"^\w\s", "", frase)
    frase = re.sub(r"\s+", " ", frase)
    return frase.strip()
```

E procuramos repetições exatas ou expressões-chave.

Por exemplo:

"boleto de quitação do meu cartão consignado"

"Isso, do meu cartão consignado."

Podemos detectar repetição de entidade/intenção:

```
r"\bcart[aão]o consignado\b"
```

duas vezes.

Seu prompt possui regra de consistência: se o cliente repetiu várias vezes a mesma informação, `at_cx_compr3` não pode ser “Sim”.

10. Transferência

Precisamos de Regex com **negação**.

Errado:

```
r"transfer"
```

Porque:

```
"não vou transferir"
```

seria marcado como transferência.

Melhor:

```
TRANSFERENCIA_NEGADA = (
    r"\b(?:n[aã]o|nem)\b.{0,40}\btransfer"
)
```

e:

```
TRANSFERENCIA_POSITIVA = (
    r"\b(?:vou|irei|preciso|posso)\b.{0,40}"
    r"\b(?:transferir|encaminhar)\b"
)
```

Motor:

```
if re.search(TRANSFERENCIA_NEGADA, texto, re.I):
    transferencia = False

elif re.search(TRANSFERENCIA_POSITIVA, texto, re.I):
    transferencia = True

else:
    transferencia = None
```

11. CES — mudança de canal

O documento exige três campos CES e define `ces1_canal` como mudança de canal.

Regex:

```
MUDANCA_CANAL = [
```

```

    r"\bentre em contato (?:com|pelo|na)\b",
    r"\blique para\b",
    r"\bacesse o site\b",
    r"\bvá at[eé] uma ag[eê]ncia\b",
    r"\bpelo aplicativo\b",
    r"\bpelo WhatsApp\b",
    r"\bcentral de atendimento\b"
]

```

No nosso atendimento:

"a senhora entrar em contato amanhã"

Mas é preciso distinguir:

mesmo canal

de:

outro canal

Aqui Regex encontra a evidência; uma regra contextual decide se efetivamente houve mudança.

12. CES — retrabalho

Podemos procurar:

```

RETRABALHO = [
    r"\bj[aá] liguei\b",
    r"\bnovamente\b",
    r"\bde novo\b",
    r"\boutra vez\b",
    r"\bj[aá] entrei em contato\b",
    r"\b\d+\s+vezes\b"
]

```

Exemplo:

"já liguei três vezes"

teria:

```
ces2_retrabalho = "Sim"
```

13. Recontato

Esse campo pode ter um detector fortíssimo.

```
RECONTATO_EXPLICITO = [
    r"\bentre em contato amanhã\b",
    r"\blique novamente\b",
    r"\bretorne amanhã\b",
    r"\bentre em contato novamente\b"
]
```

Se aparecer:

```
recontato_obrigatorio = True
```

Logo:

```
probabilidade_recontato = "Alta"
```

Nesse caso específico isso não seria uma previsão da IA.

É praticamente consequência explícita da orientação dada.

14. Reclamação

Regex pode capturar manifestações explícitas:

```
RECLAMACAO = [
    r"\bquero reclamar\b",
    r"\bquero registrar uma reclama[cç][aã]o\b",
    r"\babsurdo\b",
    r"\binsatisfeit[oa]\b",
    r"\bp[eé]ssimo\b",
    r"\bn[aã]o aguento mais\b"
]
```

Mas:

```
impl_potencial_reclamacao = Alto/Médio/Baixo
```

é uma avaliação de risco.

O documento exige cinco impactos com classificação Alto/Médio/Baixo/Não Aplicável.

Regex coleta:

```
cliente ameaçou reclamar?
cliente mencionou ouvidoria?
problema ficou sem solução?
houve recontato?
houve perda financeira?
houve contestação?
```

Depois Rule Engine/LLM classifica o risco.

15. Ouvidoria, BACEN, Procon e judicial

Aqui Regex é excelente.

```
ESCALONAMENTO = {
    "ouvidoria":
        r"\bouvidoria\b",
    "bacen":
        r"\b(?:bacen|banco central)\b",
    "procon":
        r"\bprocon\b",
    "judicial":
        r"\b(?:processo|advogado|justi[cç]a|judicial)\b",
    "reclamacao":
        r"\breclama[cç][aã]o\b"
}
```

Esses sinais podem alimentar o alerta crítico.

Seu documento define, por exemplo, risco de BACEN, risco judicial, LGPD, fraude, prejuízo financeiro e potencial alto de ouvidoria entre condições possíveis para alerta crítico.

16. Cancelamento

```
CANCELAMENTO = [
    r"\bquero cancelar\b",
    r"\bvou cancelar\b",
    r"\bcancele\b",
    r"\bcancelamento\b",
    r"\bn[aã]o quero mais\b"
]
```

Encontrar essas frases é 100% determinístico.

Classificar:

```
potencial_cancelamento = Alto
```

já depende da regra que você escolher.

17. Contestação

```
CONTESTACAO = [
    r"\bn[aã]o reconhe[cç]o\b",
    r"\bn[aã]o fiz essa compra\b",
    r"\bcontestar\b",
    r"\bcontesta[cç][aã]o\b",
    r"\bessa compra n[aã]o [eé] minha\b"
]
```

Excelente Regex para contexto bancário.

18. Fraude

```
FRAUDE = [
    r"\bfraude\b",
    r"\bgolpe\b",
    r"\bclonagem\b",
    r"\bclonado\b",
    r"\binvas[aã]o\b",
    r"\bn[aã]o reconhe[cç]o\b"
]
```

Mas novamente:

palavra "fraude" encontrada

não é igual a:

fraude comprovada

Então salve:

fraude_mencionada = True

e não:

fraude_confirmada = True

sem evidência adicional.

19. LGPD/dados sensíveis

Regex é muito forte para identificar exposição de:

CPF
RG

telefone
e-mail
CEP
conta
agência
cartão
data nascimento

Exemplo:

```
DADOS_PESSOAIS = {
    "cpf": r"\b\d{3}\.\?\d{3}\.\?\d{3}-?\d{2}\b",
    "data": r"\b\d{2}/\d{2}/\d{4}\b",
    "telefone": r"\b(?:\(?\d{2}\)\?|s*)?\d{4,5}-?\d{4}\b"
}
```

Isso serve tanto para auditoria quanto anonimização.

20. Pontos extras

O documento contém `inv_extra1`.

Uma regra específica já está no próprio prompt:

Se o cliente elogiou espontaneamente o atendimento, `inv_extra1` deve ser Sim.

Detector:

```
ELOGIOS = [
    r"\bexcelente atendimento\b",
    r"\bmuito bom atendimento\b",
    r"\bvoc[eê] me ajudou muito\b",
    r"\bparab[eé]ns\b",
    r"\bmuito obrigado pelo atendimento\b"
]
```

Isso pode ser praticamente determinístico.

21. Inaderências

Os sete critérios precisam aparecer individualmente e podem gerar penalidade de -100.

Aqui eu criaria detectores especialmente conservadores.

```
INADERENCIAS = {
    "desligamento_inadequado": [...],
    "informacao_incorreta_explicita": [...],
    "linguagem_ofensiva": [...],

```

```

        "recusa_atendimento": [...],
        ...
    }

```

Se não houver prova:

```
"classificacao": "Não"
```

ou:

```
"Não Aplicável"
```

conforme a regra oficial do critério.

Não usar LLM para “achar” uma inaderência crítica sem evidência.

22. Falhas operacionais

Seu documento exige pelo menos:

```

Sondagem inadequada
Próximos passos não informados
Validação ou registro incompleto

```

e pede que mesmo zero seja exibido.

Podemos fazer:

```

falhas = {
    "sondagem_inadequada": 0,
    "proximos_passos_nao_informados": 0,
    "validacao_incompleta": 0
}

```

Exemplo:

```

if not encontrou_proximos_passos:
    falhas["proximos_passos_nao_informados"] += 1

```

E:

```

if dados_obrigatorios_faltantes:
    falhas["validacao_incompleta"] += 1

```

Isso é Rule Engine, não precisa de LLM.

23. Cálculo das notas: ZERO LLM

Aqui eu tiraria completamente a IA.

O documento determina:

```
Sim = fator 1
Não Aplicável = fator 1
Parcial = 0,5
Não = 0
```

```
nota = peso × fator
```

Python:

```
FATORES = {
    "Sim": 1,
    "Não Aplicável": 1,
    "Parcial": 0.5,
    "Não": 0
}

def calcular_nota(peso, classificacao):
    fator = FATORES[classificacao]

    return {
        "peso": peso,
        "fator": fator,
        "nota": peso * fator
    }
```

Exemplo:

```
calcular_nota(5, "Parcial")
```

resultado:

```
{
    "peso": 5,
    "fator": 0.5,
    "nota": 2.5
}
```

24. Score Experiência: também ZERO LLM

Seu prompt determina explicitamente:

Resolutivo Sim	+30
at_cx_classifl Sim	+30
Esforço baixo	+20
Esforço médio	+10
Fricção Não	+20
Fricção N/A	+20

Reclamação Alto -50

mínimo 0
máximo 100

Python:

```
def score_experiencia(
    resolutivo,
    reincidencia,
    esforco,
    friccao,
    reclamacao
):
    score = 0

    if resolutivo == "Sim":
        score += 30

    if reincidencia == "Sim":
        score += 30

    if esforco == "Baixo":
        score += 20

    elif esforco == "Médio":
        score += 10

    if friccao in ["Não", "Não Aplicável"]:
        score += 20

    if reclamacao == "Alto":
        score -= 50

    return max(
        0,
        min(100, score)
    )
```

Ou seja:

LLM não calcula nada.

25. Score Operador: também ZERO LLM

Eu faria:

```
score_operador = (
    score_relacionamento
    + score_resolutividade
```

```

        + score_cx
    )

```

Depois penalidade:

```

if possui_inaderencia_critica:
    score_operador = 0

```

Depois bônus conforme regra do prompt.

E classificação:

```

def classificar_operador(score):

    if score == 100:
        return "Extraordinário"

    if score >= 85:
        return "Supera as expectativas"

    if score >= 70:
        return "Atende as expectativas"

    if score >= 50:
        return "Pode Melhorar"

    return "Pode Melhorar Muito"

```

Essas faixas estão explicitamente definidas no documento.

26. Validação de coerência: Python, não LLM

Essa é uma das partes mais poderosas.

Exemplo do próprio prompt:

```

if atendimento_resolutivo == "Não":
    assert probabilidade_recontato != "Baixa"

```

Outro:

```

if not confirmou_dados:
    assert at_cx_intro2 != "Sim"

```

Outro:

```

if not informou_proximos_passos:
    assert at_cx_classif2 != "Sim"

```

Outro:

```
if linguagem_ofensiva:
    assert humor_atendente != "Positivo"
```

Essas regras constam expressamente no documento.

Crie um motor:

```
def validar_coerencia(r):

    erros = []

    if (
        r["atendimento_resolutivo"] == "Não"
        and
        r["probabilidade_recontato"] == "Baixa"
    ):
        erros.append(
            "Não resolutivo incompatível com recontato baixo."
        )

    if (
        not r["confirmou_dados"]
        and
        r["at_cx_intro2"] == "Sim"
    ):
        erros.append(
            "Confirmação de dados incompatível."
        )

    return erros
```

27. Cobertura integral: também Python

Seu documento manda contar campos antes de finalizar.

Faça:

```
COBERTURA_ESPERADA = {
    "relacionamento": 14,
    "resolutividade": 2,
    "cx": 9,
    "inaderencias": 7,
    "pontos_extras": 1,
    "ces": 3,
    "friccao": 1,
    "impactos": 5,
    "causa_raiz": 4
}
```

Validador:

```
def validar_cobertura(resultado):
```

```

erros = []

for grupo, esperado in COBERTURA_ESPERADA.items():

    produzido = len(resultado[grupo])

    if produzido != esperado:
        erros.append(
            f"{grupo}: esperado {esperado}, recebido {produzido}"
        )

return erros

```

Se tiver erro:

```

if erros:
    raise ValueError(erros)

```

O relatório **não nasce**.

Então onde a LLM realmente entra?

Aqui está a divisão que eu recomendo:

Componente	Regex Python/Rules LLM	
Protocolo	☒	
CPF	☒	
CEP	☒	
Data	☒	
Telefone	☒	
Produto explícito	☒	☒
Motivo explícito	☒	☒
Validação de dados	☒	☒
Protocolo informado	☒	☒
Desculpas	☒	
Gerundismo	☒	☒
Vícios de linguagem	☒	☒
Ofensas	☒	☒
Transferência	☒	☒
Recontato explícito	☒	☒
Mudança de canal	☒	☒
Retrabalho explícito	☒	☒
Ouvidoria/BACEN/Procon	☒	☒

Componente	Regex	Python/Rules	LLM
Elogio	☒	☒	☒
Cálculo das notas		☒	☒
Penalidades		☒	☒
Bônus		☒	☒
Score Experiência		☒	☒
Score Operador		☒	☒
Classificação operador		☒	☒
Validação de coerência		☒	☒
Validação de cobertura		☒	☒
Humor	☐		☒
Empatia genuína	☐		☒
Clareza contextual	☐		☒
Proatividade	☐	☒	☒
Esforço	☐	☒	☒
Fricção	☐	☒	☒
Risco futuro	☐	☒	☒
Causa raiz	☐		☒
Responsabilidade	☐	☒	☒
Insights			☒
Coaching		☒	☒
Resumo executivo			☒
Conclusão executiva			☒

Portanto, eu não tentaria fazer:

Regex = 100% da inteligência.

Eu faria algo muito melhor:

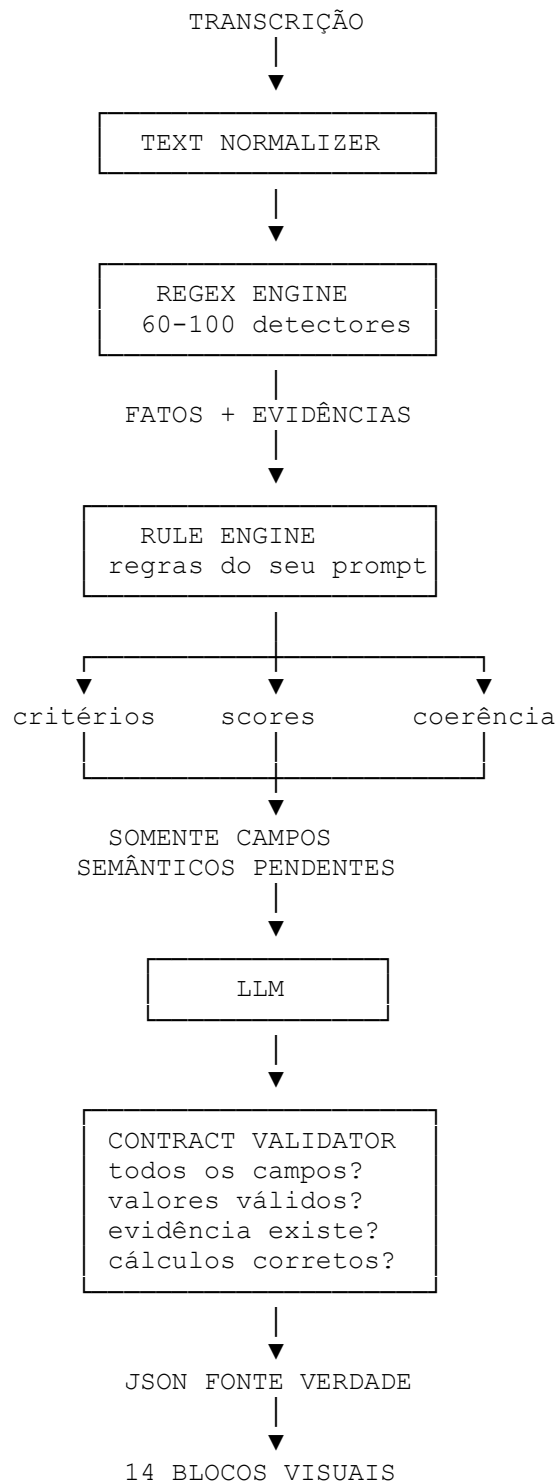
Regex + Python determinístico
 = 100% da estrutura
 + 100% dos cálculos
 + 100% das validações
 + 100% das evidências objetivas

LLM
 = somente interpretação semântica

Isso combina perfeitamente com uma das regras mais importantes do seu documento: **os dados estruturados são a fonte de verdade e o relatório visual é apenas a representação desses dados.**

E seu relatório final continua contendo obrigatoriamente os **14 blocos na ordem definida**, sem remover nenhum critério.

A arquitetura que eu adotaria, portanto, seria:



Esse desenho atende muito melhor ao espírito do seu prompt do que deixar uma LLM ler tudo e “decidir tudo”. O próprio documento exige preservação de códigos, pesos, notas, justificativas, penalidades, bônus e validações, além de proibir a apresentação de hipóteses como fatos.

O próximo passo mais útil seria transformar **esse arquivo inteiro** em uma **matriz técnica** **Critério** → **Regex** → **Regra Python** → **LLM** → **Evidência** → **Peso** → **Fórmula**, incluindo individualmente `at_rel_cord1` até `at_inad_compr7`, CES, impactos, causa raiz etc. Isso viraria praticamente a especificação de implementação do seu motor de análise.